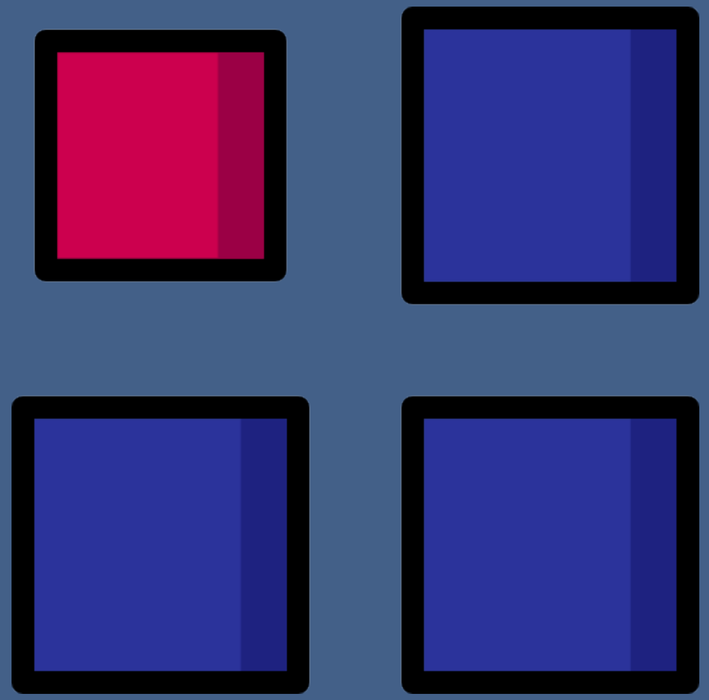
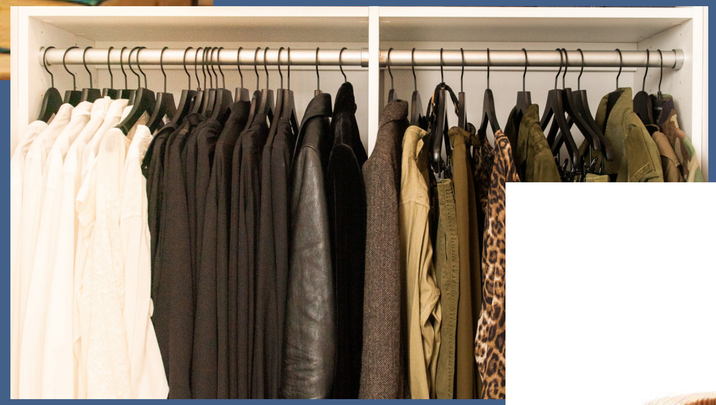
Arrays



Arrays



Arrays

- It is a box of macaroons.
- All macaroons in this box are next to each other
- Each macaroon can be identified uniquely based on their location
- The size of box cannot be changed

"a"

- Array can store data of specified type
- Elements of an array are located in a contiguous
- Each element of an array has a unique index
- The size of an array is predefined and cannot be modified

What is an Array?

In computer science, an array is a data structure consisting of a collection of elements , each identified by at least one array index or key. An array is stored such that the position of each element can be computed from its index by a mathematical formula.

Why do we need an Array?

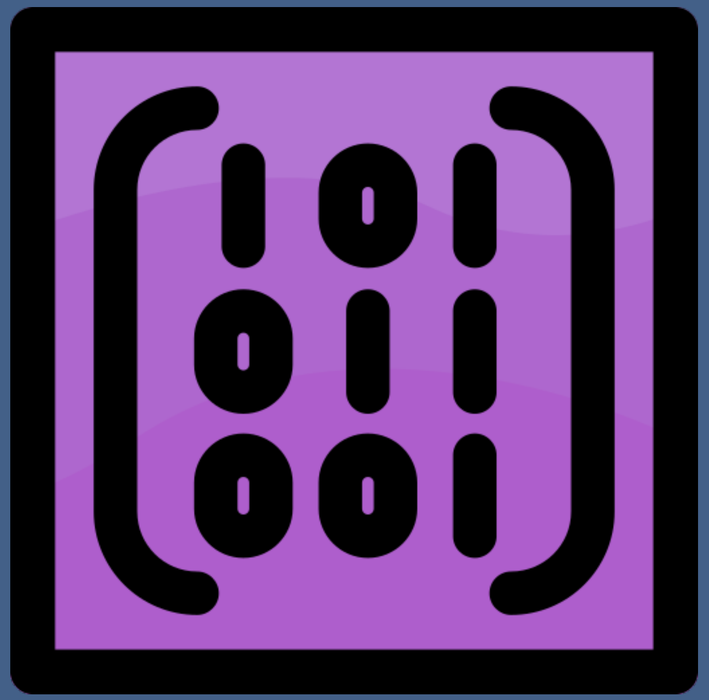
3 variables

number1
number2
number3

- What if 500 integer?
- Are we going to use 500 variables?

The answer is an ARRAY

Types of Arrays



Types of Arrays

Types of Arrays

One dimensional array : an array with a bunch of values having been declared with a single index.

a[i] —> i between 0 and n

a[2]					a[5]			
5	4	10	11	8	11	68	87	12
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

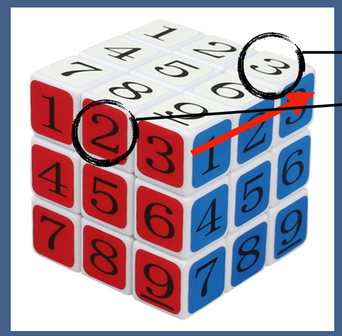
Two dimensional array : an array with a bunch of values having been declared with double index.

a[i][j] —> i and j between 0 and n

	a[0][4]						a[2][5]	
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
[0]	1	33	55	91	20	51	62	74	13
[1]	5	4	10	11	8	11	68	87	12
[2]	24	50	37	40	48	30	59	81	93

Three dimensional array : an array with a bunch of values having been declared with triple index.

a[i][j][k] —> i , j and k between 0 and n



a[2][0][2]

a[0][0][1]

Types of Arrays

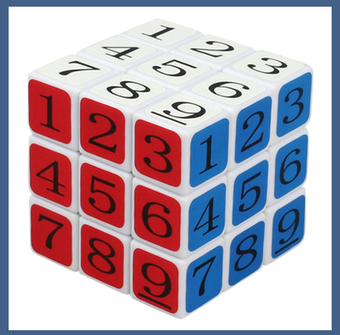
One dimensional array :

5	4	10	11	8	11	68	87	12

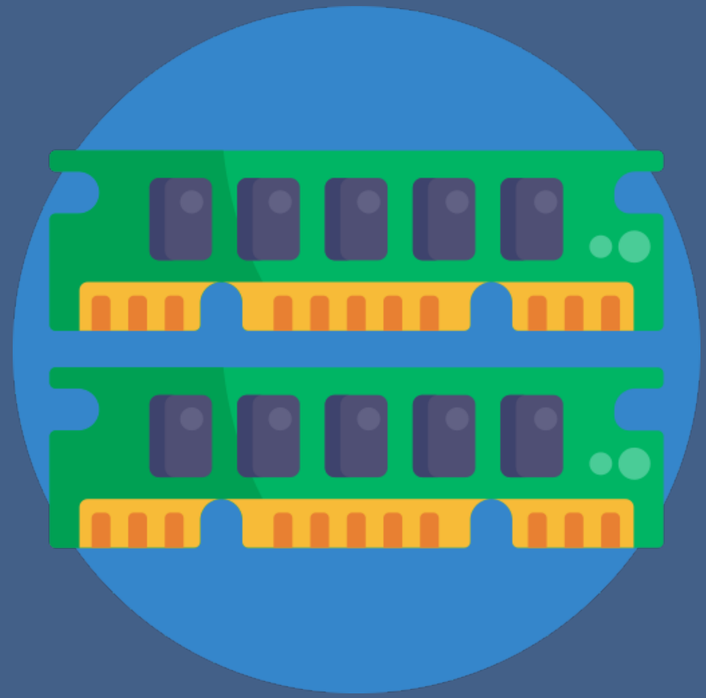
Two dimensional array :

1	33	55	91	20	51	62	74	13
5	4	10	11	8	11	68	87	12
24	50	37	40	48	30	59	81	93

Three dimensional array :



Arrays in Memory

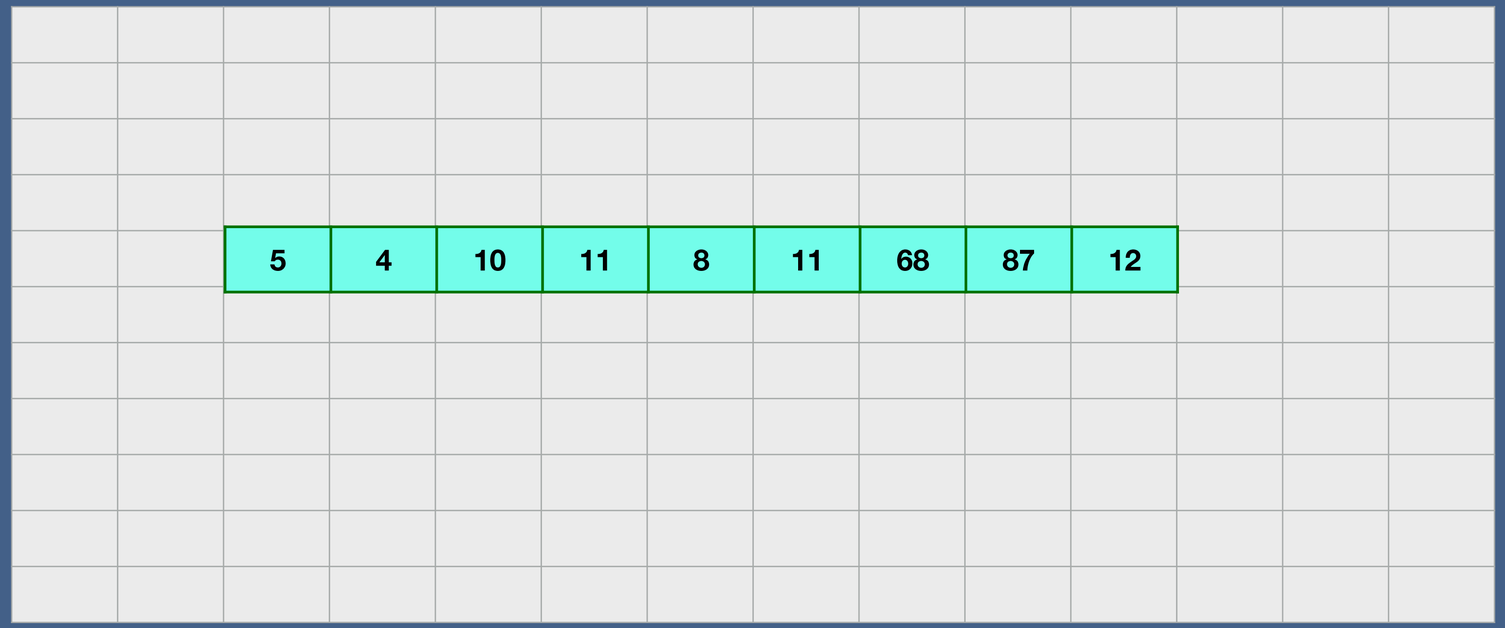


Arrays in Memory

One Dimensional

5	4	10	11	8	11	68	87	12

Memory

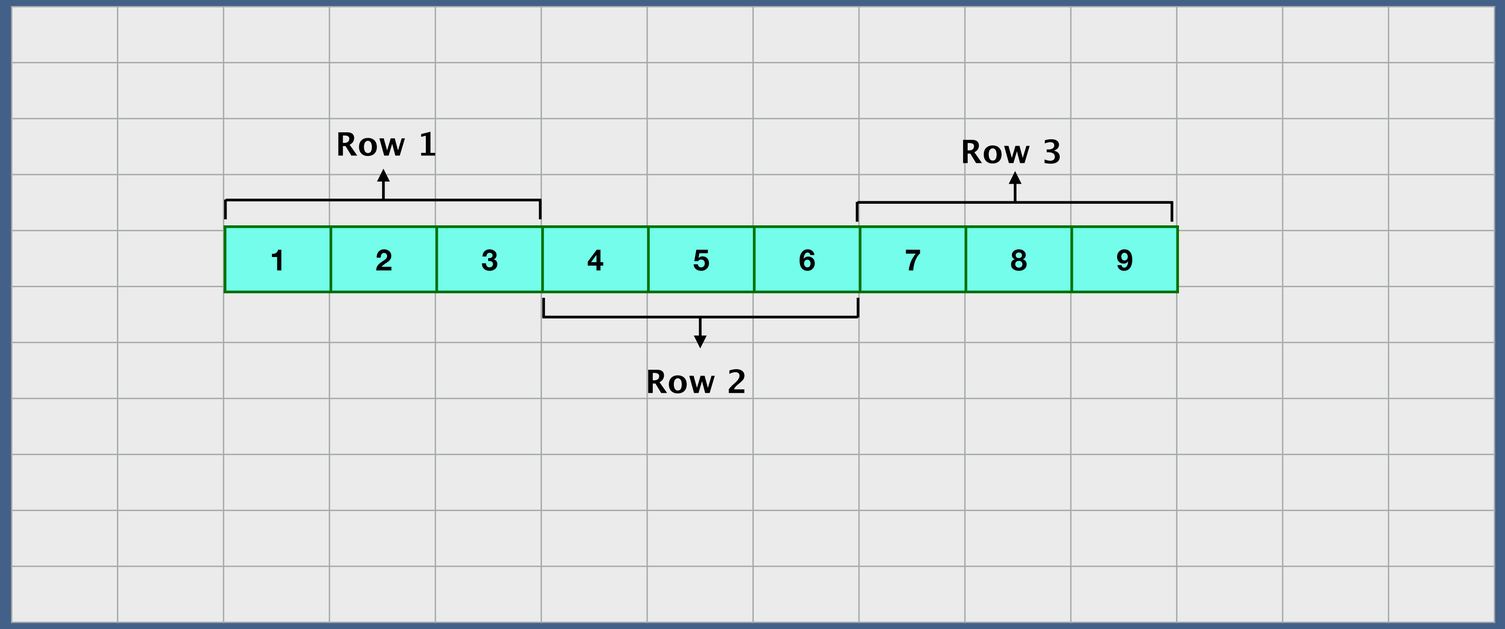


Arrays in Memory

Two Dimensional array

1	2	3
4	5	6
7	8	9

Memory

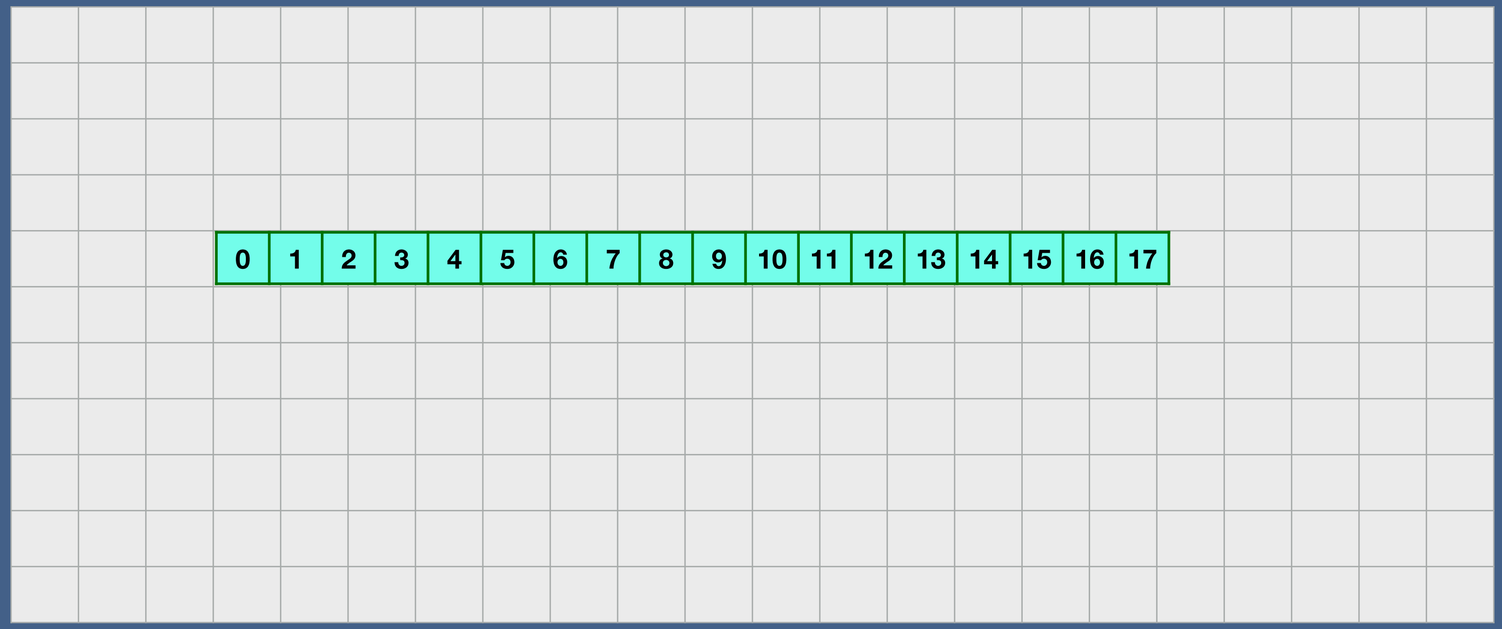


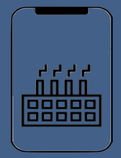
Arrays in Memory

Three Dimensional array

```
{{{ 0,  1,  2},
  { 3,  4,  5},
  { 6,  7,  8},
  { 9,  10,  11},
  { 12, 13,  14},
  { 15, 16,  17}}};
```

Memory



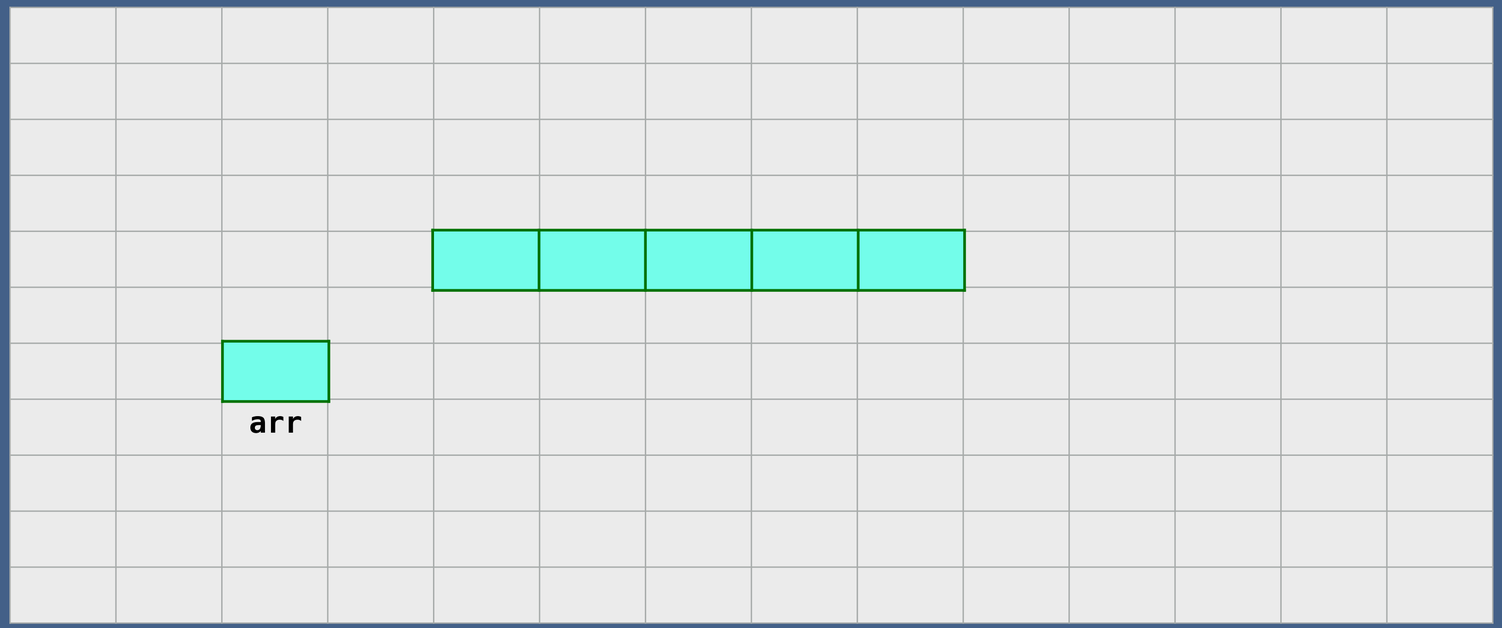


Creating an Array

When we create an array , we:

- Declare - creates a reference to array
- Instantiation of an array - creates an array
- Initialization - assigns values to cells in array

Memory

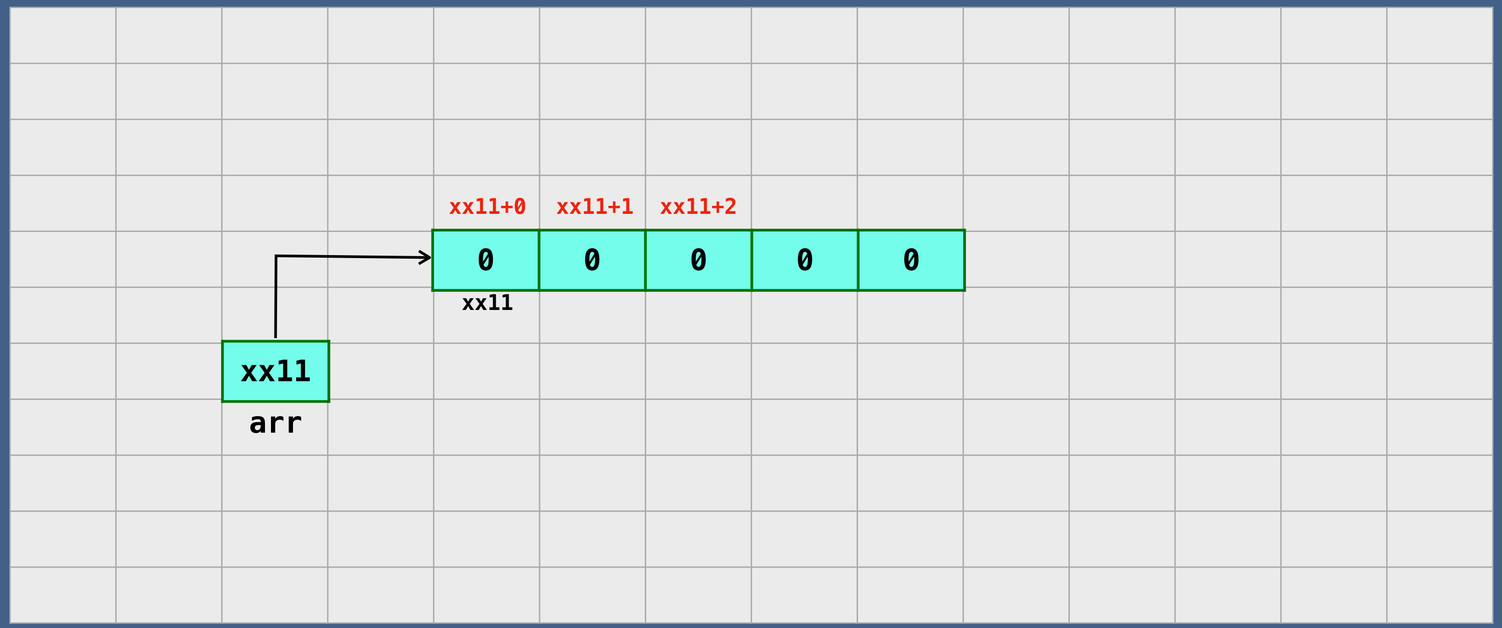


Creating an Array

When we create an array , we:

- Declare - creates a reference to array
- Instantiation of an array - creates an array
- Initialization - assigns values to cells in array

Memory

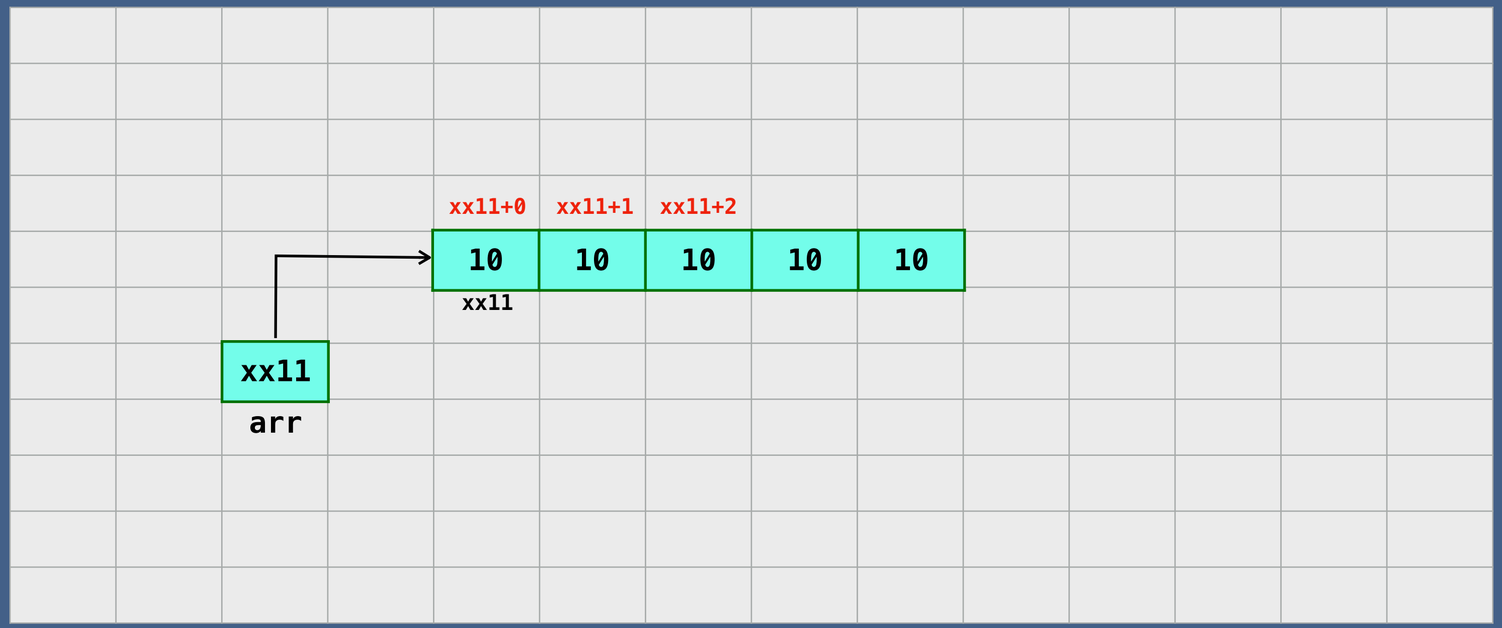


Creating an Array

When we create an array , we:

- Declare - creates a reference to array
- Instantiation of an array - creates an array
- Initialization - assigns values to cells in array

Memory



Creating an Array

When we create an array , we:

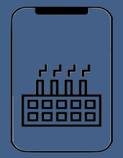
- Declare - creates a reference to array
- Instantiation of an array - creates an array
- Initialization - assigns values to cells in array

```
dataType[] arr
```

```
arr = new dataType[]
```

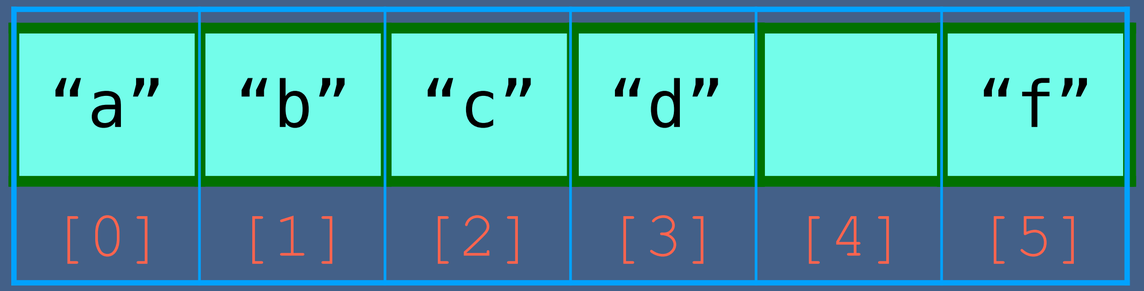
```
arr[0] = 1
```

```
arr[1] = 2
```



Insertion in Array

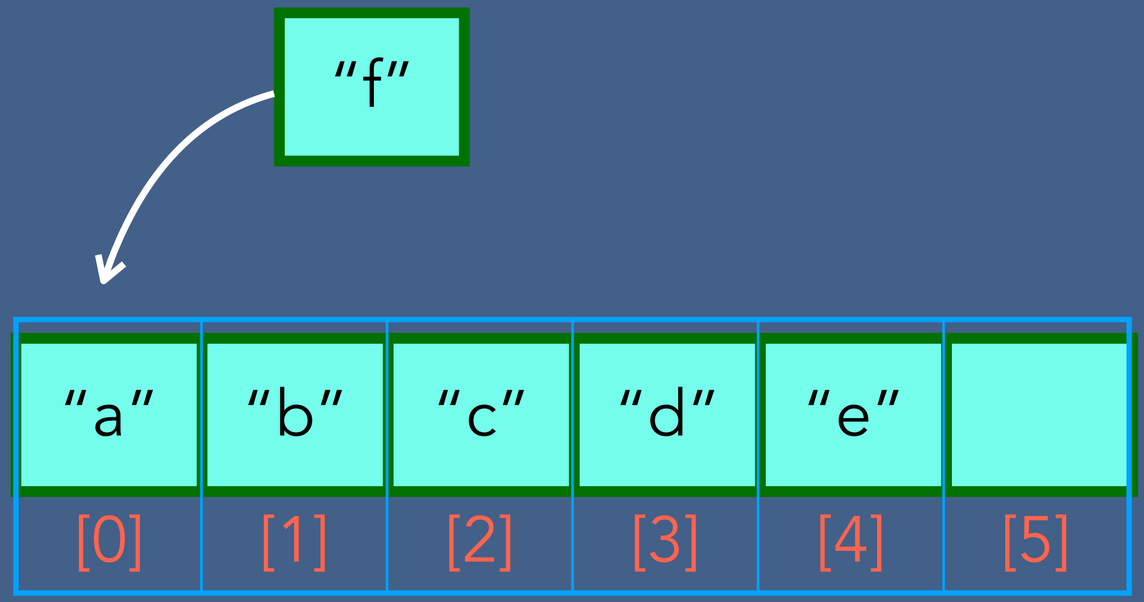
myArray =



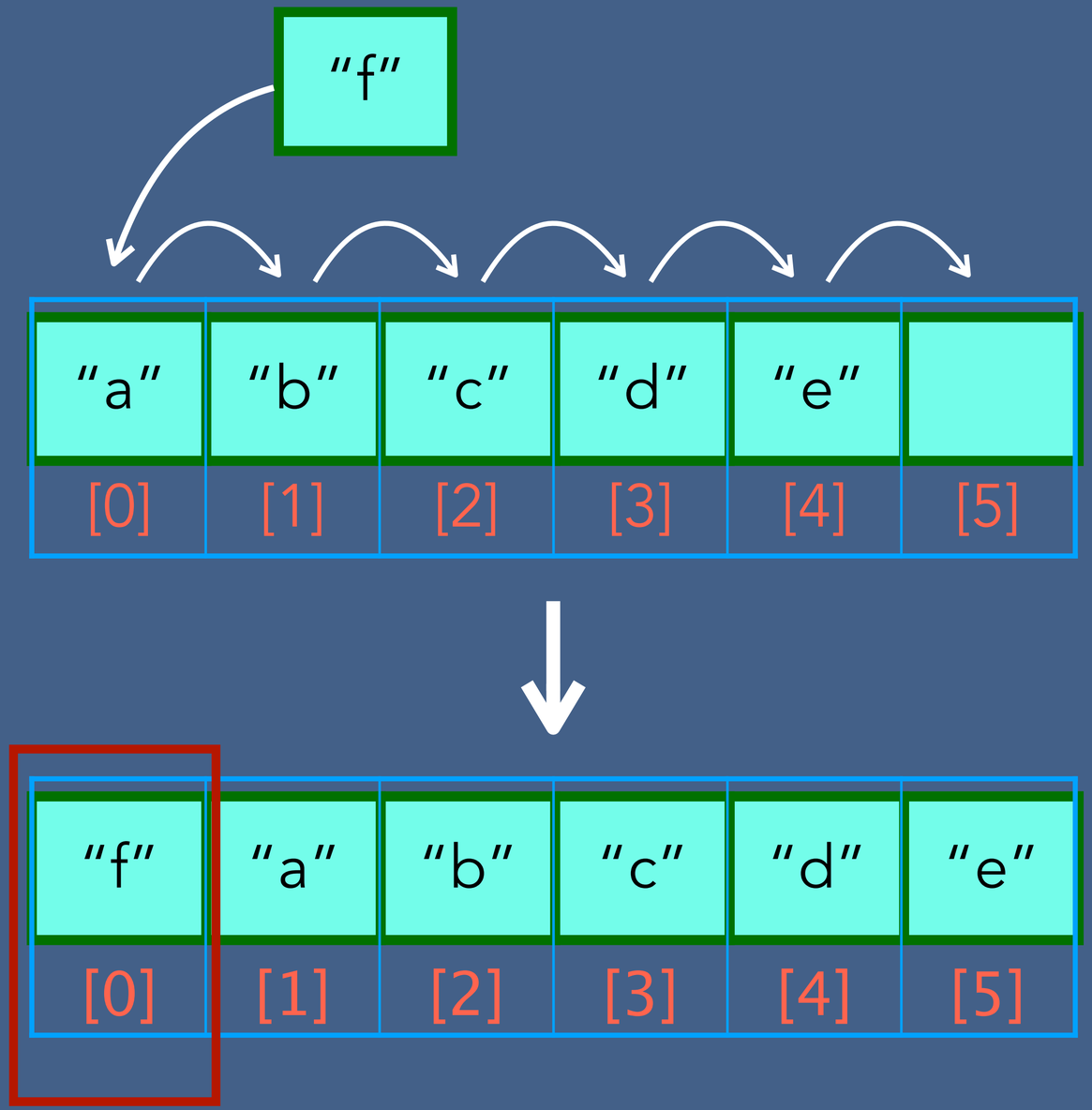
myArray[3] = “d”

myArray[5] = “f”

Insertion in Array

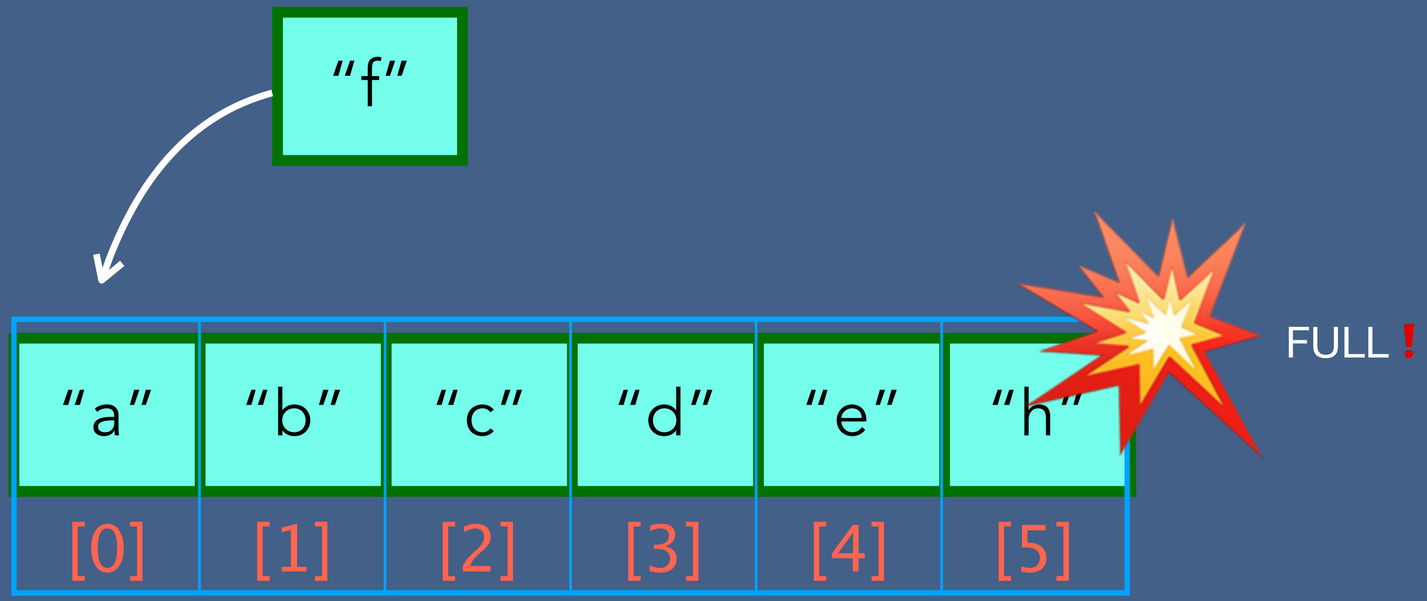


Insertion in Array



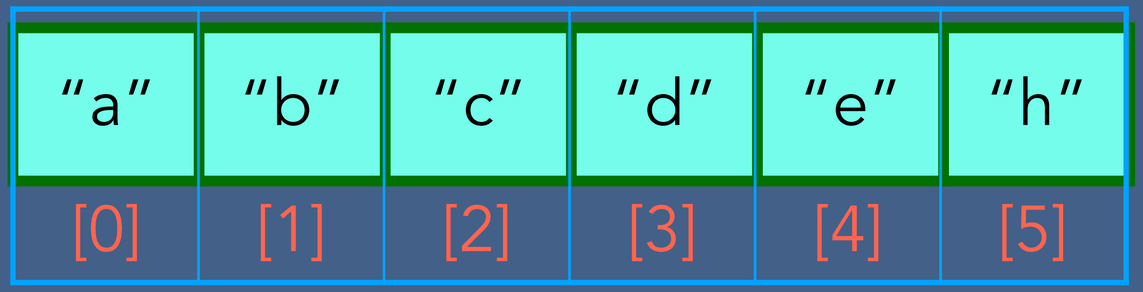
Insertion in Array

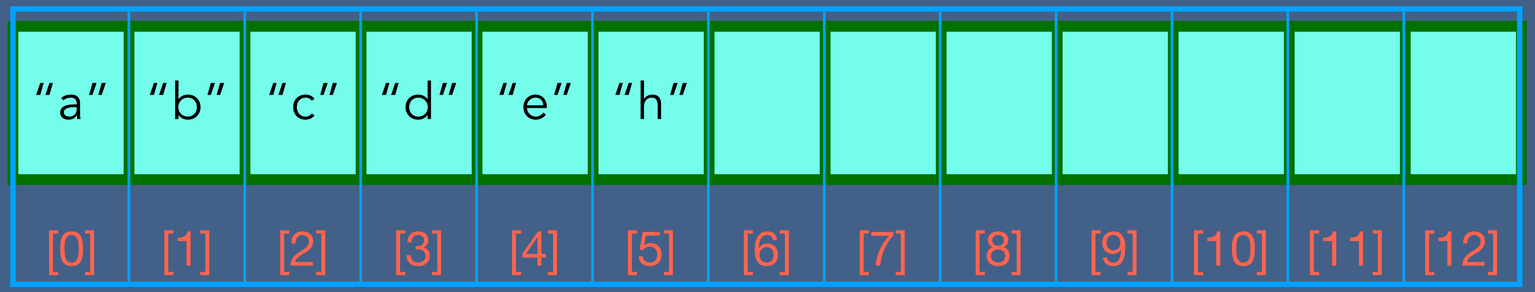
Wait A minute! What Happens if the Array is Full?



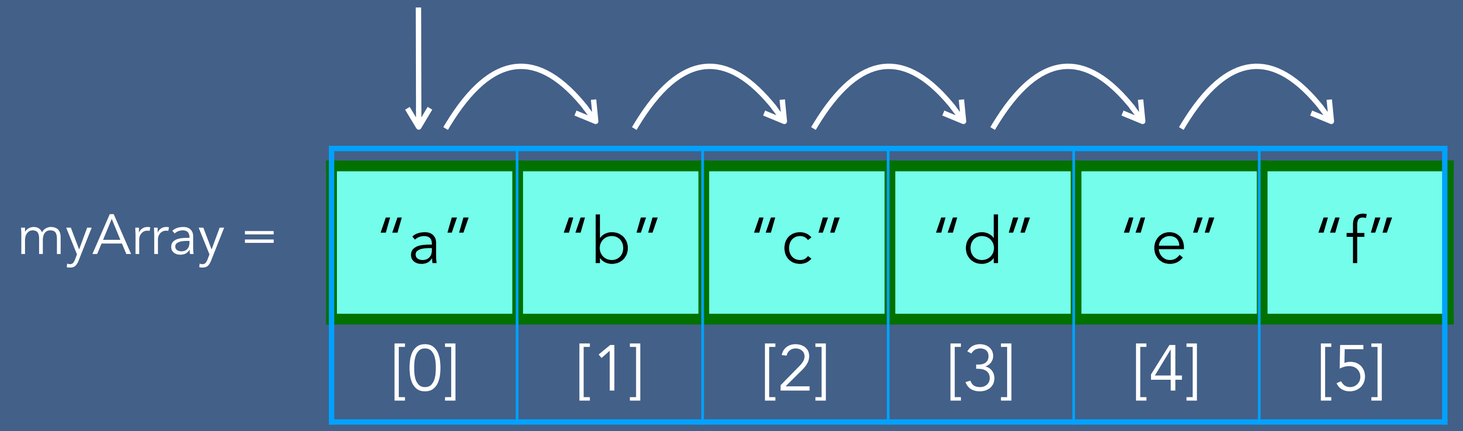
Insertion in Array

Insertion , when an array is full.





Array Traversal



myArray[0] = “a”

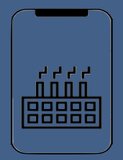
myArray[1] = “b”

myArray[2] = “c”

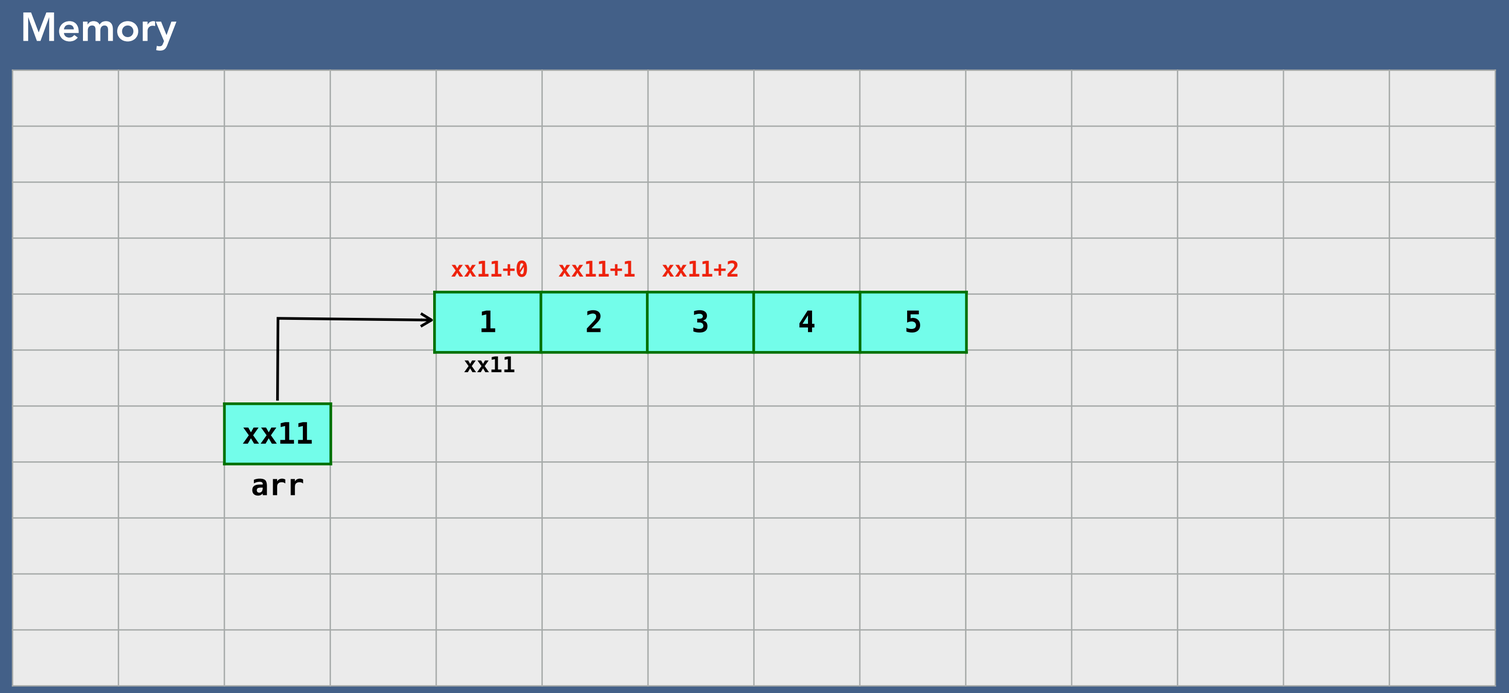
myArray[3] = “d”

myArray[4] = “e”

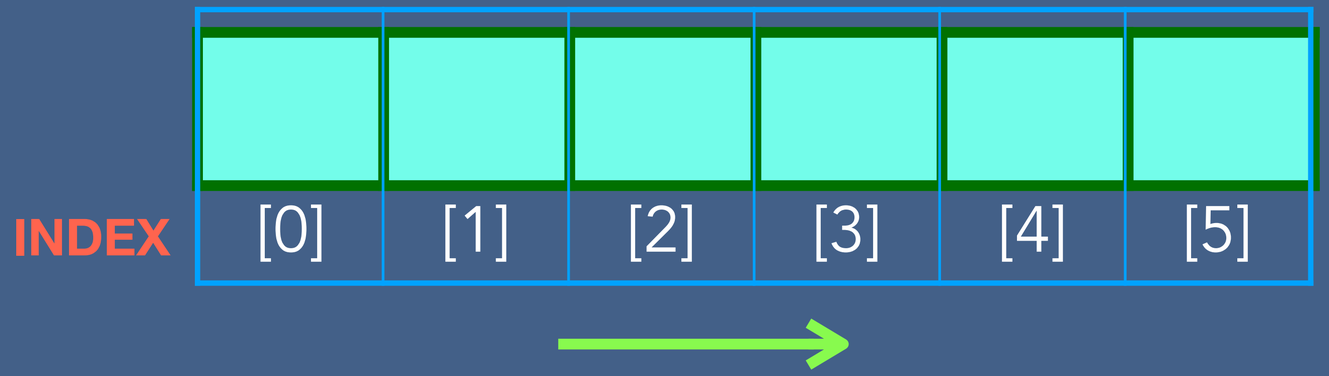
myArray[5] = “f”



Accessing Array Element



Accessing Array Element

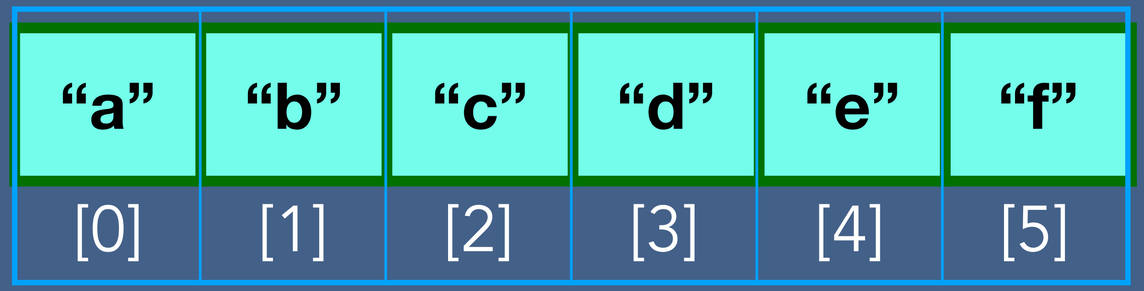


<arrayName>[index]

Accessing Array Element

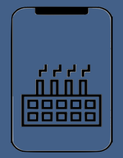
For example:

myArray =



myArray[0] = “a”

myArray[3] = “d”

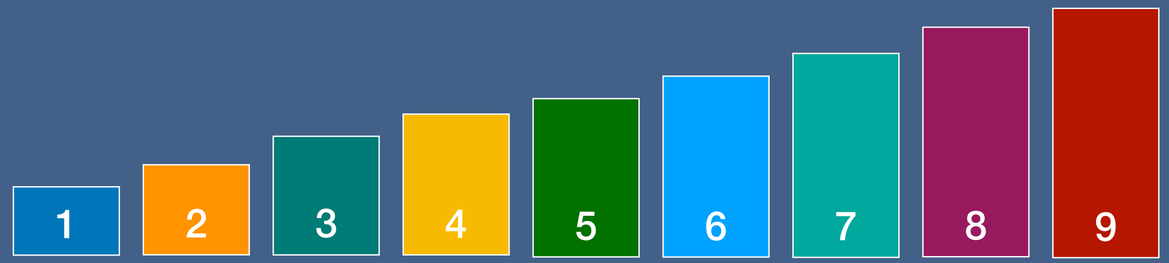


Finding Array Element

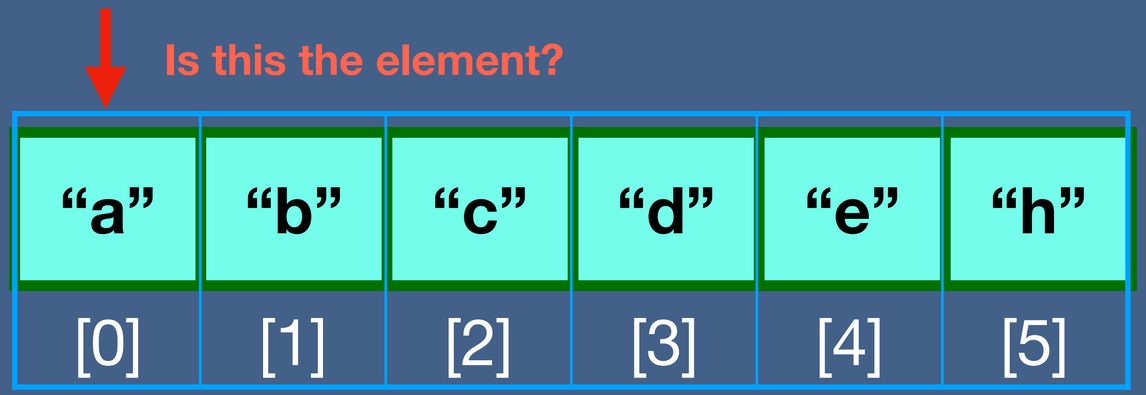
myArray =

“a”	“b”	“c”	“d”	“e”	“h”
[0]	[1]	[2]	[3]	[4]	[5]

myArray2 =



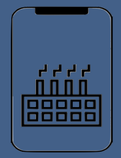
Finding Array Element



Deleting Array Element

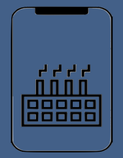
Integer.MIN_VALUE

10	**20**	-2^{31}	**40**	-2^{31}	-2^{31}
[0]	[1]	[2]	[3]	[4]	[5]



Time and Space Complexity of 1D Arrays

Operation	Time complexity	Space complexity
Creating an empty array	O(1)	O(n)
Inserting a value in an array	O(1)	O(1)
Traversing a given array	O(n)	O(1)
Accessing a given cell	O(1)	O(1)
Searching a given value	O(n)	O(1)
Deleting a given value	O(1)	O(1)

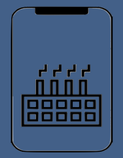


Find Number of Days Above Average Temperature

```
How many day's temperature?  2

Day 1’s high temp:  1

Day 2’s high temp:  2
```

Output

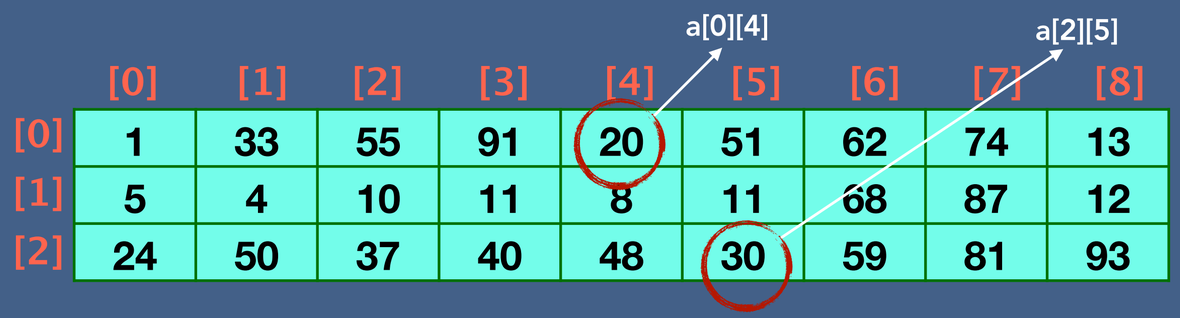
```
Average = 1.5
1 day(s) above average
```



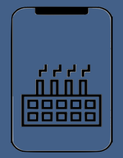
Two Dimensional Array

An array with a bunch of values having been declared with double index.

a[i][j] —> i and j between 0 and n



Day 1 - 11, 15, 10, 6
Day 2 - 10, 14, 11, 5
Day 3 - 12, 17, 12, 8
Day 4 - 15, 18, 14, 9



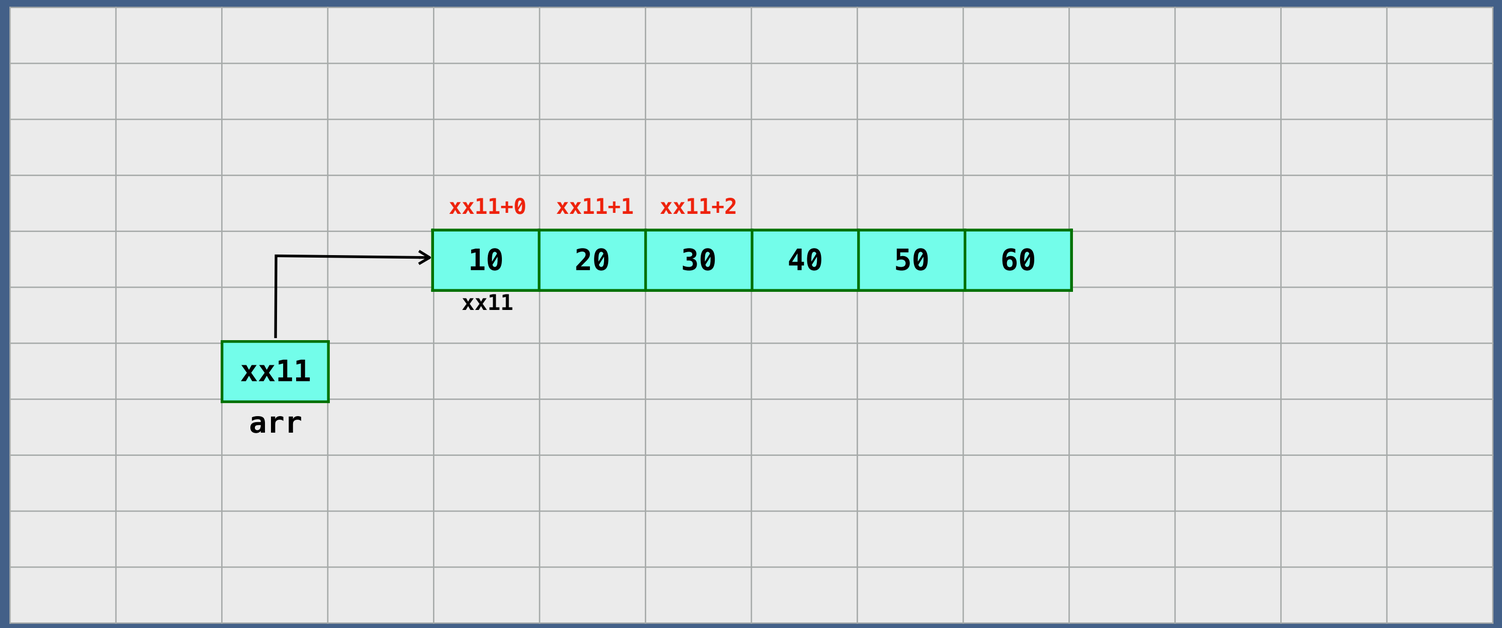
Two Dimensional Array

When we create an array , we:

- Declare - creates a reference to array
- Instantiation of an array - creates an array
- Initialization - assigns values to cells in array

10	20	30
40	50	60

Memory



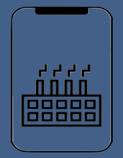
Insertion - Two Dimensional Array

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
[0]	50								
[1]									
[2]						30			

arrayName[2][5] = 30

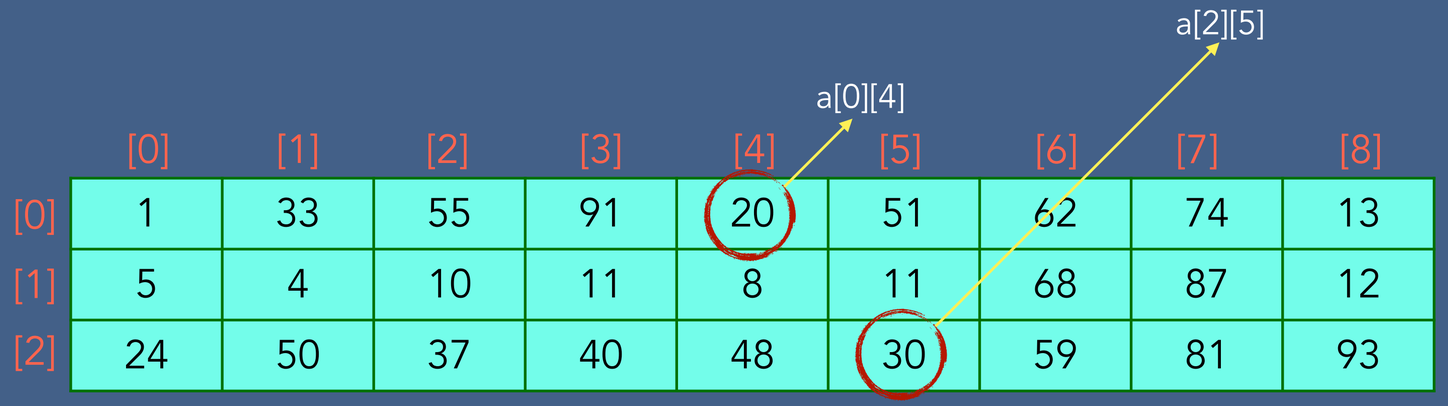
arrayName[0][0] = 50

arrayName[0][0] = 60 ⟶ Occupied



Access an element of Two Dimensional Array

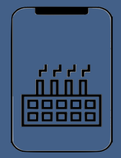
a[i][j] —> i is row index and j is column index



Traversing Two Dimensional Array

1	33	55	91
5	4	10	11
24	50	37	40

1 33 55 91 5 4 10 11 24 50 37 40



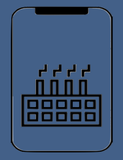
Searching Two Dimensional Array

Is this the element?

1	33	55	91
5	4	44	11
24	50	37	40

Searching for 44

The element is found at [1][2]

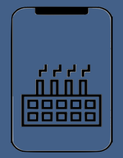


Deleting Array Element in 2D Array

	[0]	[1]	[2]	[3]
[0]	1	33	55	91
[1]	5	-2^{31}	10	11
[2]	24	50	37	-2^{31}

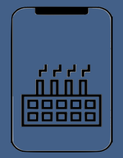
arrayName[2][3] = Integer.MIN_VALUE

arrayName[1][1] = Integer.MIN_VALUE



Time and Space Complexity of 2D Array

Operation	Time complexity	Space complexity
Creating an empty array	O(1)	O(mn)
Inserting a value in an array	O(1)	O(1)
Traversing a given array	O(mn)	O(1)
Accessing a given cell	O(1)	O(1)
Searching a given value	O(mn)	O(1)
Deleting a given value	O(1)	O(1)



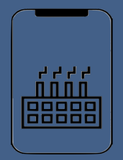
When to use/avoid Arrays

When to use

- To store multiple variables of same data type
- Random access

When to avoid

- Same data type elements
- Reserve memory



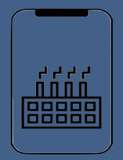
When to use/avoid Arrays

When to use

- To store multiple variables of same data type
- Random access

When to avoid

- Same data type elements
- Reserve memory



Array Project

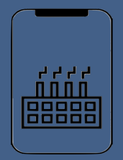
Array Project

Find Number of Days Above Average Temperature

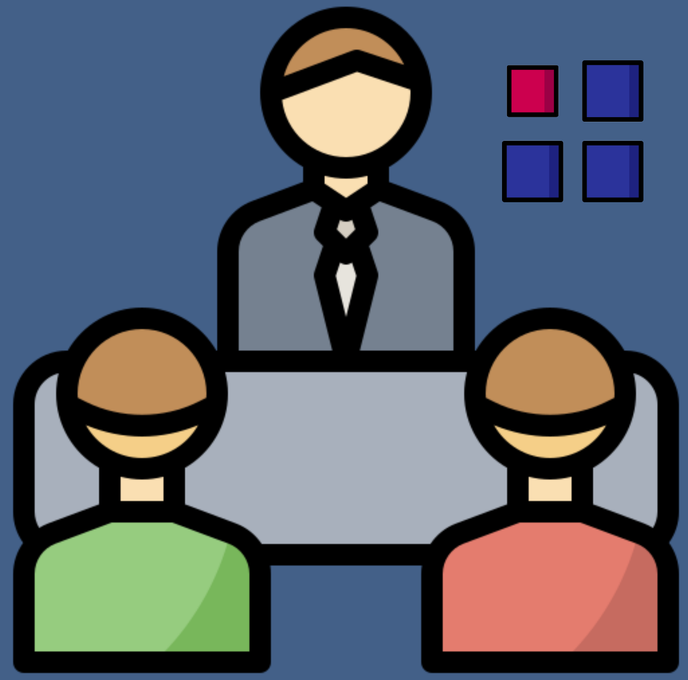
```
How many day's temperature?  2

Day 1’s high temp:  1

Day 2’s high temp:  2
```

Output

```
Average = 1.5
1 day(s) above average
```



Array Interview Questions



Missing Number

Find the missing number in an integer array of 1 to 100

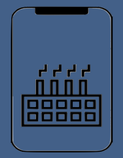
1,2,3,4,5,6…50,51,52..100

1,2,3,4,5,6,7,8,9,10 = 55

1,2,3,4,5,6,8,9,10 = 48

7

1,2,3,4,5,6…n = **n(n+1)/2**



Pairs / Two Sum

Write a program to find all pairs of integers whose sum is equal to a given number.

[2, 6, 3, 9, 11] 9 ⟶ [6,3]

1. Two Sum

Easy 18064 647 Add to List Share

Given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to* `target`.

You may assume that each input would have ***exactly* one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

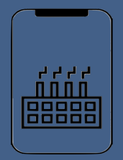
```
Input: nums = [2,7,11,15], target = 9
Output: [0,1]
Output: Because nums[0] + nums[1] == 9, we return [0, 1].
```

Example 2:

```
Input: nums = [3,2,4], target = 6
Output: [1,2]
```

Example 3:

```
Input: nums = [3,3], target = 6
Output: [0,1]
```

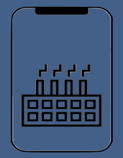


Pairs / Two Sum

Write a program to find all pairs of integers whose sum is equal to a given number.

[2, 6, 3, 9, 11] 9 ⟶ [6,3]

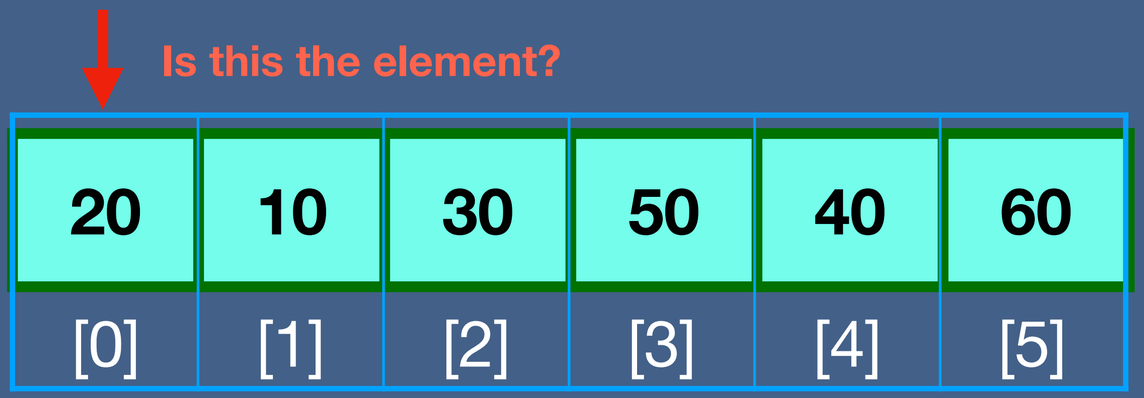
- Does array contain only positive or negative numbers?
- What if the same pair repeats twice, should we print it every time?
- If the reverse of the pair is acceptable e.g. can we print both (4,1) and (1,4) if the given sum is 5.
- Do we need to print only distinct pairs? does (3, 3) is a valid pair forgiven sum of 6?
- How big is the array?

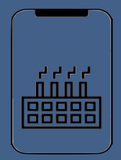


Search for a Value

Write a program to to check if an array contains a number in Java

Searching for 40 ⟶ Found at the index of 4



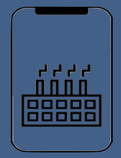


Max Product of Two Integers

Write a program to find maximum product of two integers in the array where all elements are positive

maxProduct = 0

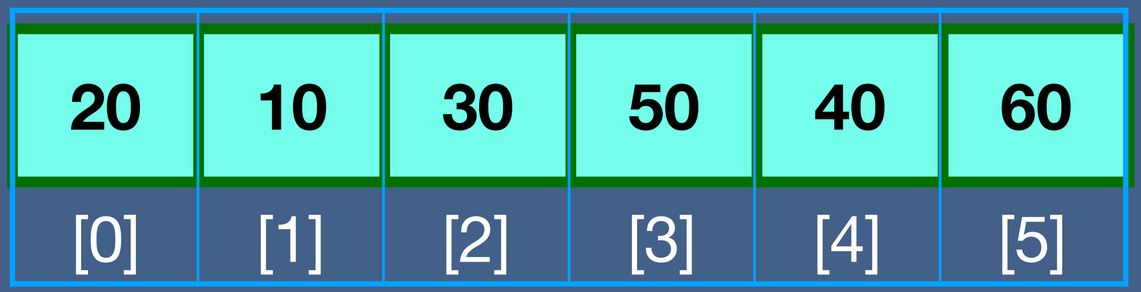
20	10	30	50	40	60
[0]	[1]	[2]	[3]	[4]	[5]



Is Unique

Write a program to check if an array is unique or not.

isUnique ⟶ True



Permutation

Your are given two integer arrays. Write a program to check if they are permutation of each other.

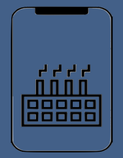
Permutation ⟶ True

2	1	3	5	4	6

1	3	2	4	6	5

sum1 = 21
sum2 = 21

mul1 = 720
mul2 = 720

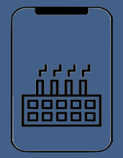


Rotate Matrix

Given an image represented by an NxN matrix write a method to rotate the image by 90 degrees.

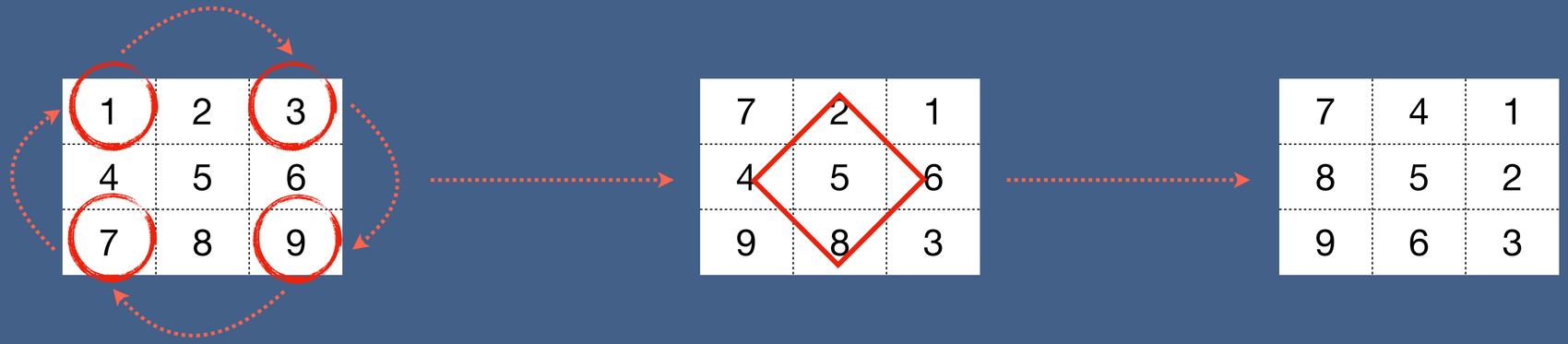
1	2	3
4	5	6
7	8	9

Rotate 90 degrees

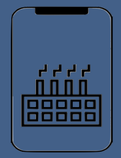


Rotate Matrix

Given an image represented by an NxN matrix write a method to rotate the image by 90 degrees.




```
for i = 0 to n
    temp = top[i]
    top[i] = left[i]
    left[i] = bottom[i]
    bottom[i] = right[i]
    right[i] = temp
```



Rotate Matrix

LeetCode Explore Day 26 Problems Mock Contest Discuss Store

Description | Solution | Discuss (999+) | Submissions

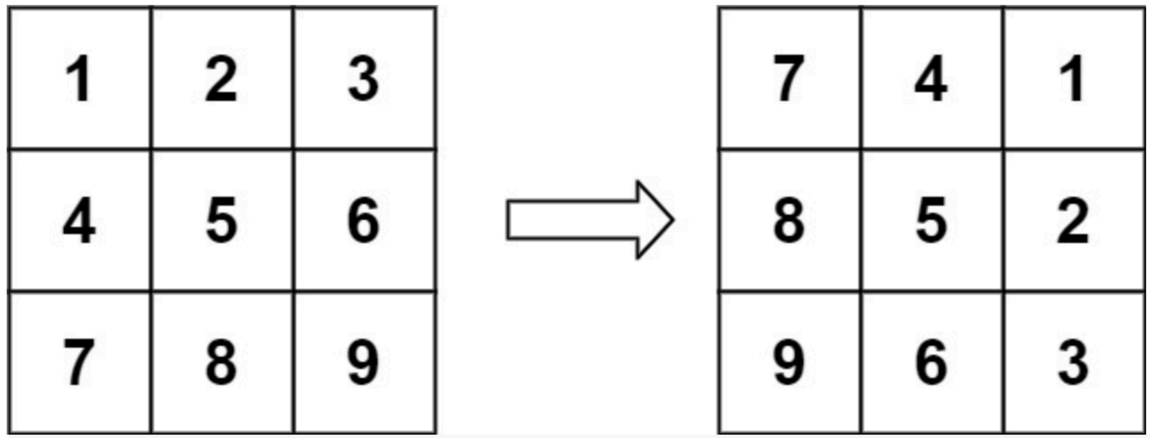
48. Rotate Image

Medium 4008 295 Add to List Share

You are given an *n* x *n* 2D `matrix` representing an image, rotate the image by 90 degrees (clockwise).

You have to rotate the image **in-place**, which means you have to modify the input 2D matrix directly. **DO NOT** allocate another 2D matrix and do the rotation.

Example 1:




```
Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [[7,4,1],[8,5,2],[9,6,3]]
```